

Raman Transition Calculator Manual

Python Translation of `Raman.txt`

Project workspace: `Raman_calculator`

May 21, 2026

Contents

1	Purpose	2
2	Source-to-Python Translation	2
3	Default Parameters	3
3.1	Temperature Mapping	3
4	Physical Model	4
4.1	Gaussian Beam Intensities	4
4.2	Effective Raman Coupling	4
4.3	Velocity and Spatial Distributions	4
4.4	Generalized Rabi Frequency	5
4.5	Transition Probability	5
5	Raman Detuning Model	5
5.1	Wave-vector and Recoil Definitions	5
5.2	Four Detuning Branches	6
5.3	Velocity Inversion	6
5.4	Calibration from Flying-up and Falling-down Scans	7
6	Graphical Interface	7
6.1	Page Structure	7
6.2	Preset Library	8
6.3	Simulation-page Action Bar	8
6.4	Primary Inputs	9
6.5	Advanced Model Parameters	9
6.6	Numerical Controls	9

6.7 Detuning-page Inputs	10
7 Numerical Method	10
8 Outputs and Interpretation	11
8.1 Rabi Oscillation Plot	11
8.2 Cloud-Size Evolution Plot	11
8.3 Derived Values Panel	11
8.4 Interactive Plot Tools	11
8.5 Locked Result Overlays	12
8.6 Detuning-page Outputs	12
9 How to Run the Software	12
10 Project File Summary	13
11 Closing Remarks	13

1 Purpose

This document describes the Python implementation of the original Mathematica script `Raman.txt`. The translated application computes Raman-driven Rabi oscillations in an atomic interferometer, displays the free-expansion growth of the transverse atom-cloud size, and integrates a second Raman-detuning workflow translated from the user’s standalone Python tool in `Raman_deturning/Raman_deturning_4.0.py`.

The implementation goals are:

- preserve the structure and default constants of the Mathematica model,
- preserve the calculation logic of the original Raman-detuning script,
- expose the main experimental parameters through a clean English-language graphical interface,
- provide direct plots for both transition probability and cloud-size evolution,
- provide a dedicated calibration workflow for reconstructing α and v_x from flying-up and falling-down detuning scans,
- document the assumptions, units, and numerical method in a form suitable for research use.

2 Source-to-Python Translation

The Mathematica source defines a model with the following components:

- Gaussian Raman-beam intensities,
- an effective Raman coupling frequency depending on the large single-photon detuning,
- a Gaussian longitudinal velocity distribution,
- a Gaussian transverse spatial distribution that broadens during free expansion,
- a two-dimensional average of the Raman transition probability over radius and velocity.

The Python code preserves these elements in `raman_model.py`. The graphical interface in `app.py` provides controlled access to the same quantities and also reports derived values such as the on-axis Raman Rabi frequency and the estimated π -pulse time.

The project now also contains an integrated detuning model in `raman_detuning_model.py`. That module preserves the branch logic of the earlier standalone Python detuning calculator and adds a calibration routine that recovers the Raman-beam angle α and transverse velocity v_x from two signed detuning measurements.

3 Default Parameters

The table below lists the translated defaults inherited from the Mathematica file.

Quantity	Default value
Recoil velocity v_{rec}	5.88×10^{-3} m/s
Initial cloud size $\sigma_r(0)$	5 mm
Large Raman detuning $\Delta/2\pi$	−1000 MHz
Fine-structure offset $\delta_2/2\pi$	157 MHz
Natural linewidth $\gamma/2\pi$	6.065 MHz
Raman beam power P_1	14.0 mW
Raman beam power P_2	5.384615 mW
Beam waist w_0	11.5 mm
Free-expansion time T in $P(T, \tau)$	56 ms
Attenuation factor	1
Coupling gain G	1

3.1 Temperature Mapping

The original Mathematica file directly specifies the transverse velocity spread as $\sigma_{v,T} = 2.65 v_{\text{rec}}$. The Python application uses the transverse temperature as the primary thermal input for that branch of the model. Therefore, the default transverse temperature is chosen such that

$$\sqrt{\frac{k_B T}{M}} = 2.65 v_{\text{rec}}, \quad (1)$$

which gives

$$T_{\text{default}} = 2.537918 \text{ } \mu\text{K}. \quad (2)$$

The GUI distinguishes two thermal directions:

- the **transverse temperature**, which sets $\sigma_{v,T}$ and therefore the radial beam convolution and cloud expansion,
- the **longitudinal temperature**, which sets $\sigma_{v,L}$ and therefore the velocity-selection distribution $f(v_L)$.

By default, the longitudinal temperature is linked to the transverse temperature so that the two directions start from the same value. The longitudinal value can then be released and edited independently whenever the experiment requires different thermal widths along and across the Raman beam axis.

4 Physical Model

4.1 Gaussian Beam Intensities

The Gaussian intensity of each Raman beam is

$$I_i(r) = \frac{2P_i}{\pi w_0^2} \exp\left(-\frac{2r^2}{w_0^2}\right), \quad (3)$$

where $i \in \{1, 2\}$, P_i is the optical power, and w_0 is the common beam waist.

4.2 Effective Raman Coupling

The translated effective Raman Rabi frequency is

$$\Omega_{\text{eff}}(r) = \frac{\gamma^2 G}{\text{attn}} \frac{\sqrt{I_1(r)I_2(r)}}{2I_{\text{sat}}} \left[\frac{5}{24\Delta} + \frac{3}{24(\Delta + \delta_2)} \right] \frac{1}{2}, \quad (4)$$

where:

- Δ is the large Raman detuning (**desacc** in the code),
- δ_2 is the offset corresponding to the additional excited-state contribution,
- $I_{\text{sat}} = 16.7 \text{ W/m}^2$,
- attn is the attenuation factor from the original script,
- G is a dimensionless gain term retained from the Mathematica implementation.

In the Python code the absolute value of the resulting coupling is used when reporting an effective frequency, because the transition probability depends on Ω_{eff}^2 .

4.3 Velocity and Spatial Distributions

The longitudinal velocity distribution is Gaussian:

$$f(v_L) = \frac{1}{\sqrt{2\pi} \sigma_{v,L}} \exp\left(-\frac{v_L^2}{2\sigma_{v,L}^2}\right). \quad (5)$$

The transverse cloud size after free-expansion time T is

$$\sigma_r(T) = \sqrt{\sigma_r(0)^2 + \sigma_{v,T}^2 T^2}. \quad (6)$$

The associated radial probability density is

$$h(r, T) = \frac{1}{2\pi\sigma_r(T)^2} \exp\left(-\frac{r^2}{2\sigma_r(T)^2}\right). \quad (7)$$

4.4 Generalized Rabi Frequency

For each (r, v_L) point, the generalized Raman Rabi frequency is

$$\Omega_R(r, v_L) = \sqrt{\Omega_{\text{eff}}(r)^2 + (k_{\text{eff}}v_L - d)^2}, \quad (8)$$

with

$$k_{\text{eff}} = \frac{4\pi}{\lambda}, \quad (9)$$

and d the user-specified two-photon detuning.

4.5 Transition Probability

The Raman transition probability is evaluated as

$$P(T, \tau) = \int \int 2\pi r f(v_L) h(r, T) \left[\frac{\Omega_{\text{eff}}(r)}{\Omega_R(r, v_L)} \sin\left(\frac{\Omega_R(r, v_L)\tau}{2}\right) \right]^2 dv_L dr. \quad (10)$$

This is a direct Python translation of the Mathematica function `Ptrans[T_, tau_, d_, w0_, attn_]`.

5 Raman Detuning Model

5.1 Wave-vector and Recoil Definitions

The integrated detuning tool uses the same conventions as the original standalone Python script. The laser wave number is

$$k = \frac{2\pi}{\lambda}, \quad (11)$$

the effective Raman wave number is

$$k_{\text{eff}} = 2k = \frac{4\pi}{\lambda}, \quad (12)$$

and the recoil term is represented as an angular frequency

$$\omega_r = 2\pi f_r, \quad (13)$$

where f_r is entered in kHz in the GUI.

5.2 Four Detuning Branches

Let v_z be the known vertical atomic velocity, α the Raman-beam angle relative to the z axis, and v_x the transverse velocity to be inferred or propagated. The transition direction controls the recoil sign through

$$s_{\text{tr}} = \begin{cases} +1, & \text{for F1} \rightarrow \text{F2}, \\ -1, & \text{for F2} \rightarrow \text{F1}. \end{cases} \quad (14)$$

The detuning model then provides four signed branches:

$$\delta_{\text{up},+} = k_{\text{eff}} v_z \sin \alpha + k_{\text{eff}} v_x \cos \alpha + s_{\text{tr}} \omega_r, \quad (15)$$

$$\delta_{\text{up},-} = -k_{\text{eff}} v_z \sin \alpha - k_{\text{eff}} v_x \cos \alpha + s_{\text{tr}} \omega_r, \quad (16)$$

$$\delta_{\text{down},+} = k_{\text{eff}} v_z \sin \alpha - k_{\text{eff}} v_x \cos \alpha + s_{\text{tr}} \omega_r, \quad (17)$$

$$\delta_{\text{down},-} = -k_{\text{eff}} v_z \sin \alpha + k_{\text{eff}} v_x \cos \alpha + s_{\text{tr}} \omega_r. \quad (18)$$

In the GUI these four cases are reported as:

- flying up, $\Delta > 0$,
- flying up, $\Delta < 0$,
- falling down, $\Delta > 0$,
- falling down, $\Delta < 0$.

The sign of the entered or computed detuning therefore matters operationally: it is used to identify which branch of the original model is active.

5.3 Velocity Inversion

When the user selects the **Detuning** $\rightarrow$ **vx** mode, the application keeps the original branch logic of the standalone script. The user provides:

- a signed detuning in kHz,
- the motion direction (**flying up** or **falling down**),
- the transition direction (**F1 $\rightarrow$ F2** or **F2 $\rightarrow$ F1**).

The sign of the detuning automatically selects either the $\Delta > 0$ or $\Delta < 0$ branch, and the program returns both the recovered v_x and the branch that was used internally.

5.4 Calibration from Flying-up and Falling-down Scans

The calibration page addresses the case in which α and v_x are both initially unknown. The user enters:

- the signed detuning extracted from the **flying up** scan,
- the signed detuning extracted from the **falling down** scan,
- the transition direction used in both scans.

Define the sign selectors

$$s_{\text{up}} = \text{sign}(\delta_{\text{up}}), \quad s_{\text{down}} = \text{sign}(\delta_{\text{down}}), \quad (19)$$

and the branch-reduced quantities

$$U = \frac{\delta_{\text{up}} - s_{\text{tr}}\omega_r}{s_{\text{up}}}, \quad D = \frac{\delta_{\text{down}} - s_{\text{tr}}\omega_r}{s_{\text{down}}}. \quad (20)$$

These satisfy

$$U = k_{\text{eff}}v_z \sin \alpha + k_{\text{eff}}v_x \cos \alpha, \quad (21)$$

$$D = k_{\text{eff}}v_z \sin \alpha - k_{\text{eff}}v_x \cos \alpha. \quad (22)$$

Therefore, the calibration step reconstructs the beam angle and transverse velocity from

$$\sin \alpha = \frac{U + D}{2k_{\text{eff}}v_z}, \quad (23)$$

$$v_x = \frac{U - D}{2k_{\text{eff}} \cos \alpha}. \quad (24)$$

This is exactly the logic implemented in the integrated Python calibration routine. After a successful calibration, the GUI can apply the recovered α back into the detuning constants and copy the recovered v_x into the calculator input field for immediate reuse.

6 Graphical Interface

The desktop interface is implemented with Tkinter and Matplotlib. The GUI is organized into two top-level pages, **Simulation** and **Detuning**, so that the full Raman-transition and detuning workflows remain available in a single application.

6.1 Page Structure

- **Simulation:** Raman transition probability, atom-cloud expansion, presets, locked overlays, and interactive plotting.
- **Detuning:** Raman detuning forward calculation, inverse v_x recovery, and calibration of α and v_x from scan results.

6.2 Preset Library

The interface now includes a preset library for rapid switching between common Raman operating conditions. Four presets are bundled with the application:

Preset	Applied values
Original Raman.txt Defaults	Direct translation of the Mathematica defaults.
Raman Down	$P_1 = 14$ mW, $P_2 = 7$ mW, $w_0 = 11.5$ mm, $T = 56$ ms, $\Delta/2\pi = -1300$ MHz.
Raman Up	$P_1 = 150$ mW, $P_2 = 70$ mW, $w_0 = 30$ mm, $T = 78$ ms, all remaining parameters inherited from the default set.
Raman Labeling	$P_1 = 150$ mW, $P_2 = 70$ mW, $w_0 = 30$ mm, $T = 780$ ms, all remaining parameters inherited from the default set.

The preset values labelled by the user as “tau = 56 ms”, “tau = 78 ms”, and “tau = 780 ms” are implemented as the advanced expansion-time parameter T , because the plotted Raman pulse duration τ remains the horizontal-axis sweep in μs .

The preset panel supports four actions:

- apply the currently selected preset and recompute the plots,
- save the current parameter set into the selected preset name,
- save the current parameter set as a new preset,
- delete a locally stored preset or a local override of a bundled preset.

User-created presets and local overrides are stored in **presets.json**.

6.3 Simulation-page Action Bar

The interface now places the main execution controls in the top-right corner of the application window whenever the **Simulation** page is active. This keeps the numerical input column focused on parameters while reserving a dedicated location for simulation management:

- **Run Simulation**: execute the current parameter set,
- **Reset to Defaults**: restore the translated **Raman.txt** defaults,
- **Export Current CSV**: export the currently active simulation only,
- **Lock Current**: preserve the current curves as reference traces,
- **Clear Locked**: remove all locked reference traces.

6.4 Primary Inputs

Field	Unit	Meaning
Transverse temperature	μK	Thermal input used to define the transverse velocity spread $\sigma_{v,T}$, the radial beam convolution, and the cloud expansion.
Use a separate longitudinal temperature	boolean	Unlocks an independent longitudinal thermal input. When disabled, the longitudinal temperature is linked to the transverse value.
Longitudinal temperature	μK	Thermal input used to define the longitudinal velocity spread $\sigma_{v,L}$ entering the velocity-selection distribution $f(v_L)$.
Large Raman detuning desacc / 2π	MHz	One-photon detuning entering the effective Raman coupling.
Beam power P_1	mW	Optical power of Raman beam 1.
Beam power P_2	mW	Optical power of Raman beam 2.
Beam waist w_0	mm	Common Gaussian waist used in the intensity model.
τ minimum	μs	Lower bound of the pulse-duration sweep.
τ maximum	μs	Upper bound of the pulse-duration sweep.
Number of τ samples	points	Plot resolution for the pulse-duration sweep.

6.5 Advanced Model Parameters

Field	Unit	Meaning
Expansion time T before Raman pulse	ms	Free-expansion time used inside the Raman-transition integral.
Initial cloud size $\sigma_r(0)$	mm	Initial transverse root-mean-square cloud size.
Two-photon detuning $d/2\pi$	kHz	Residual Raman detuning used in the generalized Rabi frequency.
Attenuation factor	a.u.	Multiplicative suppression of Ω_{eff} .
Coupling gain G	a.u.	Dimensionless gain factor retained from the original source.

6.6 Numerical Controls

Field	Unit	Meaning
Radial grid points	points	Number of quadrature points in the radial integral.
Velocity grid points	points	Number of quadrature points in the longitudinal velocity integral.
Radial cutoff	w_0 units	Upper radial integration limit in multiples of the beam waist.

Velocity cutoff	$\sigma_{v,L}$ units	Longitudinal velocity truncation used in the numerical integral.
-----------------	----------------------	--

6.7 Detuning-page Inputs

Field	Unit	Meaning
vz	m/s	Known vertical atomic velocity used in the detuning model.
alpha	deg	Raman-beam angle relative to the vertical axis. Used by the forward and inverse detuning calculators; not required as an input for calibration.
Laser wavelength	nm	Optical wavelength used to construct k and k_{eff} .
Recoil frequency	kHz	Two-photon recoil-frequency term converted internally to ω_r .
Mode selector	—	Switches between vx -> Detuning and Detuning -> vx .
Input field	mm/s or kHz	Accepts either v_x or a signed detuning, depending on the selected mode.
Motion direction	—	Specifies flying up or falling down for inverse calculations.
Transition direction	—	Specifies F1→F2 or F2→F1 .
Calibration flying-up detuning	kHz	Signed result extracted from the flying-up scan.
Calibration falling-down detuning	kHz	Signed result extracted from the falling-down scan.
Calibration transition	—	Transition direction used during the calibration scans.

7 Numerical Method

The original Mathematica script used adaptive Monte Carlo integration. In the Python translation, a deterministic tensor-product quadrature is used instead:

- the radial coordinate is sampled on a uniform grid from 0 to the user-selected radial cutoff,
- the longitudinal velocity is sampled on a uniform grid from $-\alpha\sigma_{v,L}$ to $+\alpha\sigma_{v,L}$, where α is the chosen cutoff multiplier,
- the integrals are evaluated using trapezoidal quadrature,
- the pulse-duration array is processed in chunks to keep memory usage modest while remaining fast enough for interactive use.

This method is appropriate for a responsive research tool, because the integrand is smooth and the user can directly increase the grid resolution whenever additional convergence margin is required.

8 Outputs and Interpretation

The application reports two primary plots.

8.1 Rabi Oscillation Plot

The first plot displays the transition probability $P(T, \tau)$ over the selected pulse-duration range. A dashed vertical line marks the estimated on-axis π -pulse duration whenever it lies inside the plotted window. The plot tab includes an automatic vertical-axis scaling option so that small-amplitude Raman signals are displayed with an appropriate zoom level rather than being forced onto a fixed 0 to 1 scale.

8.2 Cloud-Size Evolution Plot

The second plot displays

$$\sigma_r(t) = \sqrt{\sigma_r(0)^2 + \sigma_{v,T}^2 t^2} \quad (25)$$

from $t = 0$ to the user-selected free-expansion time T . The horizontal axis is displayed in an automatically chosen unit (μs , ms , or s) depending on the selected expansion interval.

This behavior is intentionally aligned with the original Mathematica definition `dispPos[T]`, which describes cloud broadening during free expansion prior to the Raman interaction. The same fixed interaction time T is then used inside the Raman-transition integral $P(T, \tau)$.

8.3 Derived Values Panel

The derived-values panel summarizes:

- longitudinal and transverse velocity spreads,
- initial and expanded cloud sizes,
- on-axis peak beam intensities,
- on-axis effective Raman Rabi frequency,
- estimated on-axis π -pulse time,
- numerical grid settings and integration cutoffs.

8.4 Interactive Plot Tools

The Matplotlib toolbar remains available for pan and zoom. In addition, the application provides direct in-figure interaction:

- hover readout of the nearest sampled point on either curve,
- click-to-pin markers with numerical coordinate labels,

- double-click or the **Reset Plot View** button to restore the latest auto-generated axis limits,
- a **Clear Plot Markers** button to remove pinned annotations.

8.5 Locked Result Overlays

The application can preserve the current simulation result as a locked reference while new parameter sets are computed. When multiple simulations are displayed simultaneously:

- locked curves are drawn as dashed traces,
- the current simulation is drawn as a solid trace,
- plot legends are enabled automatically,
- each legend label is generated from only the parameters that differ across the displayed simulations, keeping the legend concise while still informative.

8.6 Detuning-page Outputs

The **Detuning** page provides three text-based result areas:

- **Calculator Result:** reports either the four detuning branches or the recovered v_x and the internally selected branch,
- **Calibration Result:** reports the reconstructed α , reconstructed v_x , and the sign branch identified for both the flying-up and falling-down scan inputs,
- **Detuning Notes:** summarizes the model scope and the meaning of the sign conventions.

9 How to Run the Software

From the project directory:

```
python3 app.py
```

If dependencies need to be installed:

```
python3 -m pip install -r requirements.txt
```

To execute the smoke tests:

```
python3 -m unittest discover -s tests -v
```

To rebuild this PDF:

```
pdflatex -interaction=nonstopmode -halt-on-error \
  -output-directory=docs docs/raman_calculator_manual.tex
```

10 Project File Summary

File	Role
<code>Raman.txt</code>	Original Mathematica source retained for reference.
<code>raman_model.py</code>	Physics model, unit conversions, validation, and numerical solver.
<code>app.py</code>	Graphical desktop application with plots and parameter entry.
<code>presets.json</code>	User-written preset store containing custom presets and local overrides.
<code>tests/test_raman_model.py</code>	Numerical smoke tests for the translated model.
<code>README.md</code>	Quick-start and project summary.
<code>docs/raman_calculator_manual.tex</code>	Full technical manual.

11 Closing Remarks

This Python translation is intentionally conservative: it stays close to the original Mathematica structure, makes units explicit, and separates the physical model from the interface so that future extensions remain straightforward. Typical next steps for a laboratory workflow would be to add batch export of parameter scans, fit measured Rabi data, or expose independent longitudinal and transverse temperatures if the experiment requires a more anisotropic thermal model.